Lecture 1

Introduction and Definitions

Back in 2021, with the introduction of Copilot, the broader development community got a first taste of what it meant for the machine to automatically write code for you. Since then, programming tools based on large language models (LLMs) have only grown in capabilities; just a year after Copilot, in 2022, AlphaCode [1] was already claiming to surpass the median competitor in a programming competition. All of these advances have been made possible by the rapid progress in LLMs. But LLMs by themselves are only part of the story. Code presents unique challenges that derive from the extreme precision required to write correct programs — a few characters can be the difference between a correct program and one that contains a dangerous vulnerability. But code also presents unique opportunities; unlike other tasks for which LLMs have shown promise, code benefits from precise semantics, which allow us to test it and to reason formally about its behavior.

This course aims to introduce students to the broad field of program synthesis, including techniques based on large language models and reinforcement learning, but also symbolic techniques with complementary capabilities. The goal is to provide a comprehensive view of the modern program synthesis toolkit, with an emphasis on the benefits and limitations of different techniques, to allow practitioners to pick the best combination of tools for a specific task. But before diving into algorithms, this lecture aims to provide some historical context and to define the field of program synthesis.

1.1 What is program synthesis

The dream of automating software development has been present from the early days of the computer age. Already back in 1945, as part of his vision for the Automatic Computing Engine, Alan Turing argued that

> *Instruction tables will have to be made up by mathematicians with computing experience and perhaps a certain puzzle-solving ability…*
>
> *This process of constructing instruction tables should be very fascinating. There need be no real danger of it ever becoming a drudge, for any processes that are quite mechanical may be turned over to the machine itself. [2]*

Traditionally, the way automation was incorporated into software development was through the use of compilers and high-level languages. When the

first FORTRAN compiler was developed, it was touted as "The FORTRAN Automatic Coding System", with its goal being nothing less than to allow the IBM 704 to

> *"code problems for itself and produce as good programs as human coders (but without the errors)" [3]*

.

Compilation and synthesis are very closely related in terms of their goals: they both aim to support the generation of software from a high-level description of its behavior. In general, though, we expect a synthesizer to do more than translate a program from one notation to another as traditional compilers do; we expect it to discover *how* to perform the desired task. The line can be blurry, though, since some aggressive optimizing compilers can be argued to actually discover how to perform a computation that was specified at a higher level of abstraction; autoparallelization is one such example, where the compiler seeks to discover how to parallelize a set of operations that have been described sequentially. Historically, one distinguishing feature between a compiler and a synthesizer was the use of *search*; however, this distinction has also become less clear in recent years, with many compilers leveraging search techniques to optimize the generated code, and with the advent of Large Language Models that can synthesize entire programs without any explicit search.

Another class of techniques that is closely associated with synthesis is *declarative programming*, and in particular *logic programming*. The dream of logic programming was that programmers would be able to express the requirements of their computation in a logical form, and when given an input, the runtime system would derive an output that satisfied the logical constraints through a combination of search and deduction. So the goals are also closely related to program synthesis, but there are some important distinctions. First, rather than trying to discover an algorithm to solve a particular problem, logic programming systems rely on a generic algorithm to search for a solution to every problem. This means that for many problems, they can be dramatically slower than a specialized program. Additionally, if the problem is under-specified, the user may get a solution that is very far from that which was expected.

Finally, the field of machine learning itself forms a third class of approaches that are closely related to program synthesis. The canonical problem in machine learning is finding a predictor (i.e., a function) $f : \mathcal{X} \to \mathcal{Y}$ whose behavior closely matches a given dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$. In some sense, these datapoints (x_i, y_i) can often be thought of as "input-output" pairs, in which case machine learning becomes a form of program synthesis, where we seek the program f that (perhaps imperfectly) maps each input x_i to its output y_i. However, there are some important distinctions between program synthesis and machine learning. Perhaps the biggest one is that in machine learning, the

space of functions under consideration is typically restricted to those that adhere to a particular structure. For example, we may assume that f is a linear function, or a decision tree, or a neural network with a fixed architecture. By contrast, a core goal of program synthesis is to discover the structure that is needed to solve a particular problem, whether it involve loops, branches, or recursion. (Although, as we will see later, we will still need to impose other forms of constraints on the space of programs.) Relatedly, another distinction is that because the function space is so tightly prescribed in machine learning, each class of functions has its own set of highly specialized (and optimized) algorithms. Meanwhile, program synthesis typically takes a broader view, leading to algorithms that are (in principle) more general. Finally, traditionally there was a very important distinction in that program synthesis aspired to discovering programs that *always precisely* matched the specification. This has not been the case in machine learning, where the notions of learning from noisy data and using real-valued measures of success has been deeply ingrained in the literature from the very start through the languages of probability, statistics, and optimization. However, this distinction is somewhat less relevant today, since there is growing interest within the synthesis community in algorithms that are robust to noise, or that behave well in the presence of incomplete or informal specifications.

In addition to thinking of machine learning as a form of program synthesis, which perhaps is more of an illuminating exercise than an idea from which we can immediately derive practically useful techniques, recent years have seen a sharp increase in the use of machine learning techniques to *support* general program synthesis. In particular, the use of pre-trained Large Language Models (LLMs) such as GPT-4, Claude, and Gemini to support program synthesis has been one of the most significant developments in the field over the past five years. We will learn more about this in Unit 2, where we will cover the essential techniques that allow LLMs to synthesize programs and the strengths and weaknesses of learning-based techniques relative to symbolic approaches.

1.2 A working definition of program synthesis

So if program synthesis is not compilation, it is not logic programming, and it is not machine learning, then what is program synthesis? As mentioned before, different people in the community have different working definitions of what they would describe as program synthesis, but I believe the definition below is one that both captures most of what today we understand as program synthesis and also excludes some of the aforementioned classes of approaches.

Program Synthesis correspond to a class of techniques that are able to generate a program from a collection of artifacts that establish semantic and syntactic requirements for the generated code.

There are two elements of this definition that are important. The first is an emphasis on the generation of a program; we expect the synthesizer to produce code that solves our problem, as opposed to relying on extensive search at runtime to find a solution for a particular input, as logic programming systems do. The second is the emphasis on supporting specification of both semantic and syntactic requirements. We expect synthesis algorithms to provide us with control over the intended behavior of the generated programs, but also over how they look like — for example, what components they use, how they are structured and other non-functional aspects that we consider important.

It is important to emphasize that individual synthesis systems may not themselves provide this flexibility; in fact, a number of successful applications of program synthesis have involved specialized domains where constraints on the space of programs have been "baked in" to the synthesis system. Nevertheless, even if the flexibility is not exposed to the users, the underlying synthesis algorithms do have significant flexibility in how the space of programs is defined, and this is a big differentiator both with respect to compilation and with respect to machine learning. In general, both of these requirements imply that our synthesis procedures will rely on some form of search, although the success of synthesis will be largely driven by our ability to avoid having to exhaustively search the exponentially large space of programs that arise for even relatively simple synthesis problems.

It is also important to note that our definition of what constitutes a program is very specific, as we will see in Lecture 2, but it is broad enough to include not just programs in standard programming languages such as C, Java, or Python, but also other kinds of symbolic representations, such as mathematical expressions or even formal mathematical proofs.

1.3 Program Synthesis Today

In 2025, chances are that many of you have heard of or even used program synthesis, although you may not have known it by that name. Large Language Models (LLMs) have become an integral part of the software development process, whether it be through a web interface such as ChatGPT, Claude or Gemini, auto-complete on steroids in your favorite IDE with Github Copilot, Tabnine or Cursor, or even end-to-end coding "agents" such as Claude Code and Cursor's Agent mode. Indeed, for many of us, the fact that we can now generate a piece of code by simply describing it in natural language has become something that we almost take for granted. Beyond consumer level systems, research-level models have achieved some impressive accomplishments through the use of massive amounts of search and compute. For example, in 2022 the AlphaCode system from DeepMind demonstrated performance comparable to that of the median human participant in a competitive programming contest [1], and

shortly after, the AlphaTensor paper [4] showed that it was possible to use language models to discover new algorithms for matrix multiplication that were faster than the best known algorithms at the time.

But there is more to program synthesis than LLMs. As powerful as they are, LLMs still have some important limitations: they come with weak-to-nonexistent guarantees about the code they generate; they can be difficult to adapt to new domains; they require enormous amounts of data and infrastructure to train, and consume significant energy and compute when deployed.

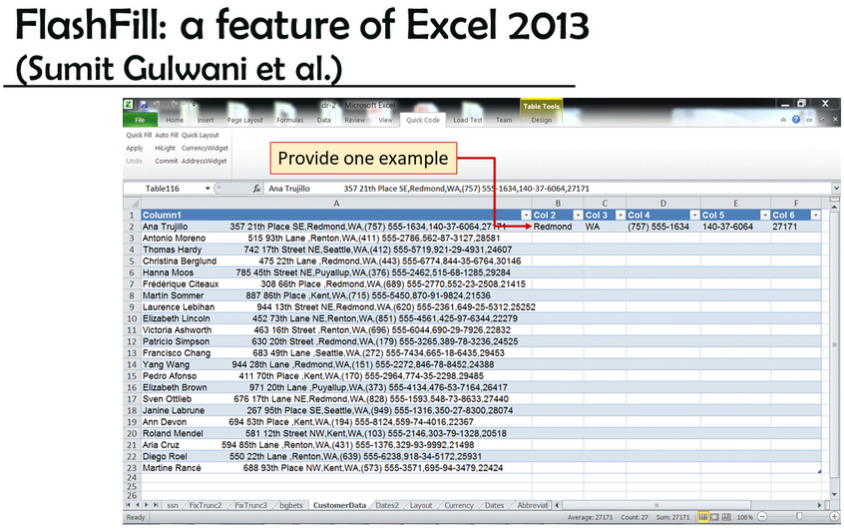
Before the advent of LLMs, the focus of the field was on efficient search techniques that could explore large spaces of possible programs to find one that satisfied a set of requirements. Those techniques were limited to synthesizing fairly small programs, and could not take advantage of unstructured means of specification such as natural language. Despite these limitations, these techniques achieved some impressive results. For example, early success stories included the ability to synthesize Karatsuba big-integer multiplication [5], Strassen's matrix-multiplication [6], or the functional cartesian product algorithm of Barron and C. Strachey, which is considered the first functional pearl [7]. The search-based techniques proved to be very effective for things like bit-vector manipulations. The winner of a program synthesis competition back in 2019 was able to synthesize every bit-vector manipulation that the organizers threw at it. Search-based techniques were also designed to work well with verification, enabling the synthesis of provably correct implementations of fairly complex algorithms; in a few years, the field was able to move from things like sorting and list reversal to algorithms and data-structure manipulations such as insertion into red-black trees or binary heaps [8].

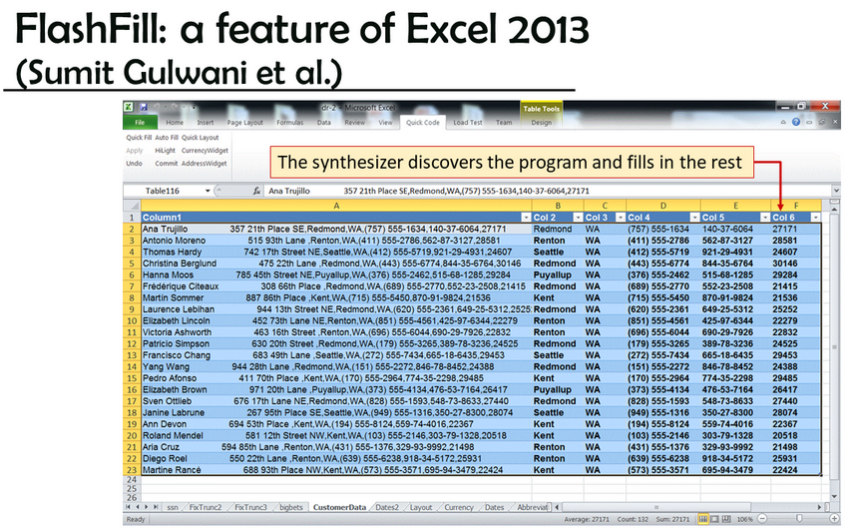
Amidst all the hype, one would be forgiven for thinking that the advent of LLMs would mean that all of these techniques have now become obsolete. In fact, they are perhaps more relevant than ever. One reason why is that search-based program synthesis techniques by themselves fill an important niche in the synthesis landscape, one in which LLMs cannot (yet) compete. In stark contrast to LLMs, which are trained on large amounts of data and require significant compute resources to run, search-based techniques can be engineered to be extremely efficient and effective as long as they are tailored to a specific domain. There are many specialized applications where training data is either unavailable, or where the cost of running an LLM is simply prohibitive, in which case search-based techniques are still the best option. Another, perhaps more important, reason is that search-based techniques are also being used to improve the performance of LLMs themselves. Indeed, as we will see towards the end of Unit 2, many techniques that have been developed by the program synthesis community are already being used to improve the performance of LLMs, and to make them more reliable and easier to use even outside of the

context of generating programs. So, despite the success of LLMs, program synthesis remains an active area of research with research papers being published every year in all the major programming systems conferences (PLDI, POPL, OOPSLA), as well as in formal methods (CAV, TACAS) and machine learning (NeurIPS, ICLR, ICML).

1.4 Program Synthesis Applications

One of the most obvious uses of program synthesis is as a software engineering aid. This is an application with which most of you will already be familiar, as noted earlier. However, there are other applications of program synthesis that may perhaps not be as obvious, but which have proven to be very impactful.

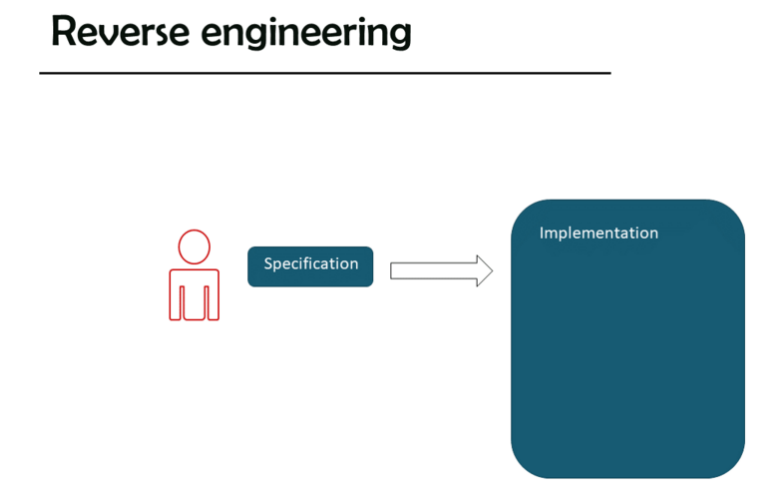


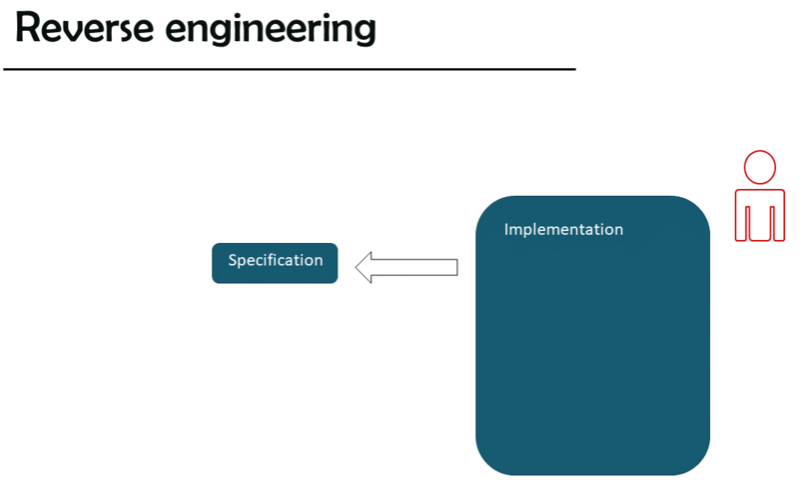


One important application has been in support of end-user or non-expert programming. Before we had chat interfaces in every desktop application, we had *FlashFill* [9]. FlashFill was introduced in Excel 2013 and was the first commercial application of program synthesis. FlashFill allows users to manipulate data by providing a few examples, and automatically derives a small program consistent with the examples and then applies the program to the rest of your data. The success of FlashFill led to an outpouring of research in end-user-programming for a variety of tasks ranging from synthesizing complex database queries from examples [10] or natural language [11], to synthesizing programs for data extraction from the web [12, 13]. This area has proven to be a good fit for program synthesis because on the one hand, there is a strong need to make data analysis and cleaning accessible to non-programmers, and on the other hand, the programs in question are generally small and easy to describe through examples or other forms of natural interaction.

Another area that has seen major interest is the reverse engineering of code. Traditionally, we think of synthesis as starting with a specification and generating an implementation from that. But in this case, that paradigm is flipped on its head: starting from an implementation, the goal is to infer a specification that characterizes the behavior of the given implementation. The idea was first proposed by Susmit Jha, Sumit Gulwani, Sanjit Seshia and Ashish Tiwari [14]. The idea was further popularized by Alvin Cheung in an approach known as *Verified Lifting*, where the goal is to discover a high-level representation that is provably equivalent to an implementation and that can be used to generate a more efficient version of the code. The idea was first applied to the problem of generating SQL queries that are equivalent to a piece of imperative code [15] or the modernization of legacy HPC applications [16]. It has continued to be succesfully applied to domains such as Map Reduce programs [17] and tensor

computations [18]. With the advent of LLMS, it became possible to make the approach more efficient and robust [19].





Verified Lifting

Synthesis based reverse engineering can help with optimization

Recent work with by Alvin Cheung and Shoaib Kamil





Java to SQL

```
List getUsersWithRoles () {
  List users = User.getAllUsers();
  List roles = Role.getAllRoles();
  List results = new ArrayList();
  for (User u : users) {
       for (Role r : roles) {
         if (u.roleId == r.id)
            results.add(u); }}
  return results; }
```

SELECT * FROM user

SELECT * FROM role

convert to

```
List getUsersWithRoles () {
  return executeQuery(
   "SELECT   u FROM user u, role r   WHERE    u.roleId == r.id
      ORDER BY u.roleId, r.id"; }
```

The models generated this way can also be used to help program analysis reason about libraries for which the source code may not be accessible. For example, a paper by Jinseong Jeon et al. [20] showed that it was possible to use synthesis to create models of complex reactive frameworks such as the Android and Java Swing by recording traces of the interaction between the framework and a test application and then forcing the synthesizer to produce a model that conforms to that trace and that follows known design patterns. A similar idea was used by Heule, Sridharan and Chandra to synthesize models of array manipulation routines in JavaScript [21].

The application of synthesis to reverse engineering is not limited to reverse engineering programs. Back in 2013, a group led by Bodik and Koskal [22] demonstrated the use of synthesis for inferring models of regulatory networks

from experimental data. The idea was to think of the regulatory network as a program, and the results of experiments as observations on the behavior of the program, so the goal was to synthesize programs that were consistent with these observations. A notable aspect of this paper was that in addition to generating an interpretable model of the regulatory network, it could synthesize *multiple* models that were all consistent with the data, and could then use automated testing techniques to suggest new experiments that would be guaranteed to disambiguate between these alternative models. A similar approach has been used in other domains such as understanding the morphology and phonology rules of natural languages [23], and more recently, under the term *symbolic regression* it has gained significant popularity in some scientific domains as a way to build interpretable models from data [24, 25]. The advent of LLMs has enabled this model building process to support larger and more complex models, as was demonstrated by the WorldCoder system [26].

1.5 Challenges

The advent of LLMs has transformed the field of program synthesis, allowing us to attack larger and more complex programs and to support a wider variety of specifications which have expanded beyond the traditional formal specifications and examples to include natural language and even visual inputs. As we will see in this course, the field is now mature enough that we can leverage off-the-shelf tools for many synthesis applications, and even problems for which the existing off-the-shelf solutions do not work can usually be attacked by bringing together the algorithmic building blocks that we cover in this course.

But despite these advances, there still remain a number of open challenges, whether one's goal is to support software engineering, or to synthesize programs to serve as interpretable models. Back in 2018 we proposed to group the challenges of Machine Programming into three pillars [27], and while the technology has advanced significantly since then, these pillars remain a useful grouping of the challenges we will be discussing in this course.

Intention. The first challenge is what we have termed the Intention challenge: how do the users tell you their goals? The definition of synthesis talks about semantic and syntactic constraints, but the exact form of these will influence all subsequent decisions about the synthesis system. Historically, early successes such as the FlashFill [9] system popularized the use of input-output examples as a means of specification. Examples come with a number of advantages, such as the ability to treat the examples as a set of unit tests against which to validate your code, but there are many tasks for which the rigidity and verbosity of input-output examples makes them unsuitable. Indeed, the rise of the language modeling paradigm has led most recent work to instead adopt natural language as the primary means of specification, but natural language

lacks precision and is difficult to automatically check for correctness.

No matter what format the specifications are given in, one big aspect of the intention challenge is how to cope with under-specification. Ultimately, the only way to completely and unambiguously characterize a program is by writing down the program itself (although perhaps not in a form that is immediately executable by your machine). Thus, typically in program synthesis we are almost always dealing with a situation in which there are *multiple* programs that satisfy the requirements. How can we tell which one the user actually wants? Of course one solution is to simply ignore this problem; if the user provides a partial specification, they have no right to complain if they get a different program from the one they wanted. In practice, though, making a good choice can make the difference between a system that is useful, and one that is not.

Invention.

Once we know what the user wants, the second challenge is to actually discover a piece of code that will satisfy those requirements. Arguably this is the central challenge of synthesis, as it potentially involves inventing new algorithmic solutions to a problem. One of the key questions we will be dealing with in this course are different techniques that the community has developed to tackle the inherent complexity of this task.

It is important to note that while LLMs have demonstrated impressive capabilities in this regard, even solving programming competition problems that require significant algorithmic ingenuity, they are not a magic bullet. They exhibit significant deficiencies when it comes to problems farther out their training distribution, as well as generating solutions unfamiliar domains-specific languages, and often require prohibitive amounts of compute to search for a solution.

Adaptation

The canonical view of synthesis is that the user is creating a brand new algorithm from scratch, and wants to leverage a synthesizer to create a correct implementation of the desired algorithm. However, most software development involves working in the context of existing software systems, fixing bugs, optimizing code, and performing other kinds of maintenance tasks. This pillar deals with the question of synthesis in a broader context, and the application of synthesis ideas to broader software development tasks beyond green-field software creation. There are a number of compelling applications of program synthesis in support of the broader software development process, especially debugging and optimization.

References

[1] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL https://www.science.org/doi/abs/10.1126/science.abq1158.

[2] B.J. Copeland. *Alan Turing's Electronic Brain: The Struggle to Build the ACE, the World's Fastest Computer*. OUP Oxford, 2012. ISBN 9780199609154. URL https://books.google.com/books?id=YhQZnczOS7kC.

[3] J. W. Backus, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hughes, and R. Nutt. The fortran automatic coding system. In *Papers Presented at the February 26-28, 1957, Western Joint Computer Conference: Techniques for Reliability*, IRE-AIEE-ACM '57 (Western), pages 188–198, New York, NY, USA, 1957. ACM. doi: 10.1145/1455567.1455599. URL http://doi.acm.org/10.1145/1455567.1455599.

[4] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.

[5] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. URL http://people.csail.mit.edu/asolar/papers/thesis.pdf.

[6] Saurabh Srivastava, Sumit Gulwani, and Jeffrey S. Foster. From program verification to program synthesis. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '10, pages 313–326, New York, NY, USA, 2010. ACM. ISBN 978-1-60558-479-9. doi: 10.1145/1706299.1706337. URL http://doi.acm.org/10.1145/1706299.1706337.

[7] John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, pages 229–239, New York, NY, USA, 2015. ACM. ISBN 978-1-4503-3468-6. doi: 10.1145/2737924.2737977. URL http://doi.acm.org/10.1145/2737924.2737977.

[8] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program synthesis from polymorphic refinement types. *SIGPLAN Not.*, 51(6):522–538, June 2016. ISSN 0362-1340. doi: 10.1145/2980983.2908093. URL http://doi.acm.org/10.1145/2980983.2908093.

[9] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, pages 317–330, New York, NY, USA, 2011. ACM. ISBN 978-1-4503-0490-0. doi: 10.1145/1926385.1926423. URL http://doi.acm.org/10.1145/1926385.1926423.

[10] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. Synthesizing highly expressive sql queries from input-output examples. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, pages 452–466, New York, NY, USA, 2017. ACM. ISBN 978-1-4503-4988-8. doi: 10.1145/3062341.3062365. URL http://doi.acm.org/10.1145/3062341.3062365.

[11] Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. Sqlizer: Query synthesis from natural language. *Proc. ACM Program. Lang.*, 1 (OOPSLA):63:1–63:26, October 2017. ISSN 2475-1421. doi: 10.1145/3133887. URL http://doi.acm.org/10.1145/3133887.

[12] Shaon Barman, Sarah E. Chasins, Rastislav Bodík, and Sumit Gulwani. Ringer: Web automation by demonstration. In *Proceedings of the 30th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '16)*, pages 748–764. ACM, 2016. doi: 10.1145/2983990.2984020. URL https://dl.acm.org/doi/10.1145/3022671.2984020.

[13] Jeevana Priya Inala and Rishabh Singh. Webrelate: Integrating web data with spreadsheets using examples. *Proceedings of the ACM on Programming Languages*, 2(POPL):28:1–28:28, 2018. doi: 10.1145/3158090. URL https://dl.acm.org/doi/10.1145/3158090.

[14] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 1*, ICSE

'10, pages 215–224, New York, NY, USA, 2010. ACM. ISBN 978-1-60558-719-6. doi: 10.1145/1806799.1806833. URL http://doi.acm.org/10.1145/1806799.1806833.

[15] Alvin Cheung, Armando Solar-Lezama, and Samuel Madden. Optimizing database-backed applications with query synthesis. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 3–14, 2013. doi: 10.1145/2462156.2462180. URL http://doi.acm.org/10.1145/2462156.2462180.

[16] Shoaib Kamil, Alvin Cheung, Shachar Itzhaky, and Armando Solar-Lezama. Verified lifting of stencil computations. *SIGPLAN Not.*, 51(6): 711–726, June 2016. ISSN 0362-1340. doi: 10.1145/2980983.2908117. URL http://doi.acm.org/10.1145/2980983.2908117.

[17] Maaz Bin Safeer Ahmad and Alvin Cheung. Automatically leveraging mapreduce frameworks for data-intensive applications. In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD '18, pages 1205–1220, New York, NY, USA, 2018. ACM. ISBN 978-1-4503-4703-7. doi: 10.1145/3183713.3196891. URL http://doi.acm.org/10.1145/3183713.3196891.

[18] Jie Qiu, Colin Cai, Sahil Bhatia, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. Tenspiler: A verified lifting-based compiler for tensor operations. In *38th European Conference on Object-Oriented Programming (ECOOP 2024), LIPIcs*, volume 313, pages 32:1–32:28, 2024. doi: 10.4230/LIPIcs.ECOOP.2024.32. URL https://doi.org/10.4230/LIPIcs.ECOOP.2024.32.

[19] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. Verified code transpilation with LLMs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 331–351, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/48bb60a0c0aebb4142bf314bd1a5c6a0-Abstract-Conference.html.

[20] Jinseong Jeon, Xiaokang Qiu, Jonathan Fetter-Degges, Jeffrey S. Foster, and Armando Solar-Lezama. Synthesizing framework models for symbolic execution. In *Proceedings of the 38th International Conference on Software Engineering*, ICSE '16, pages 156–167, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-3900-1. doi: 10.1145/2884781.2884856. URL http://doi.acm.org/10.1145/2884781.2884856.

[21] Stefan Heule, Manu Sridharan, and Satish Chandra. Mimic: Computing models for opaque code. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, pages 710–720, New

York, NY, USA, 2015. ACM. ISBN 978-1-4503-3675-8. doi: 10.1145/2786805.2786875. URL http://doi.acm.org/10.1145/2786805.2786875.

[22] Ali Sinan Köksal, Yewen Pu, Saurabh Srivastava, Rastislav Bodík, Jasmin Fisher, and Nir Piterman. Synthesis of biological models from mutation experiments. In Roberto Giacobazzi and Radhia Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, pages 469–482. ACM, 2013. doi: 10.1145/2429069.2429125. URL https://doi.org/10.1145/2429069.2429125.

[23] Kevin Ellis, Adam Albright, Armando Solar-Lezama, Joshua B. Tenenbaum, and Timothy J. O'Donnell. Synthesizing theories of human language with bayesian program induction. *Nature Communications*, 13(5024), 2022. doi: https://doi.org/10.1038/s41467-022-32012-w. URL https://www.nature.com/articles/s41467-022-32012-w.

[24] Miles Cranmer, Alvaro Sanchez Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho. Discovering symbolic models from deep learning with inductive biases. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 12345–12355, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/c9f2f917078bd2db12f23c3b413d9cba-Abstract.html.

[25] Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression.jl, 2023. URL https://arxiv.org/abs/2305.01582.

[26] Hao Tang, Darren Key, and Kevin Ellis. Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 70148–70212. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/820c61a0cd419163ccbd2c33b268816e-Paper-Conference.pdf.

[27] Justin Gottschlich, Armando Solar-Lezama, Nesime Tatbul, Michael Carbin, Martin Rinard, Regina Barzilay, Saman P. Amarasinghe, Joshua B. Tenenbaum, and Tim Mattson. The three pillars of machine programming. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL-PLDI 2018, Philadelphia, PA, USA, June 18-22, 2018*, pages 69–80, 2018. doi: 10.1145/3211346.3211355. URL http://doi.acm.org/10.1145/3211346.3211355.